



UNIVERSITÉ CATHOLIQUE DE LOUVAIN

Projet LINFO1114 - Blockchain

Auteurs

Mathéo GATTO
NOMA: 59672400

Teoman BASAR
NOMA: 55652400

Encadrement

Professeur : Van Roy Peter

Assistants:

Julien GOURGUE
Théodore COUSIN

May 9, 2026

1 Implémentation de système Blockchain

L'implémentation repose sur la fonction récursive `ExecuteBlockchainRec`, qui effectue la majeure partie du travail. Elle traite les transactions et construit la blockchain en maintenant l'état global, le bloc courant et les accumulateurs. La fonction `ExecuteBlockchain` initialise les paramètres et renvoie le résultat final.

1.1 Main Recursive block

Cas de base : s'il n'y a plus de transactions, on finalise le bloc courant et on retourne les accumulateurs. Changement de bloc : si le numéro change, on finalise le bloc puis on appelle récursivement avec le nouvel en-tête. Même bloc : chaque transaction est enrichie avec l'attribut effort, validée, puis ajoutée et appliquée si elle est valide ; sinon, elle est rejetée avant de passer à la suivante.

1.2 Helper functions

FinalizeCurrentBlock Assemble les transactions de `CurrTxs`, calcule le hash du bloc, crée le bloc final et le valide avec `ValidationBloc`. Si le bloc est valide, il est ajouté à `ChainAcc` et l'état est mis à jour ; sinon, le bloc est rejeté et l'état précédent est restauré. Les transactions sont également remises dans le bon ordre. **ApplyTransaction** Met à jour l'état (`State`) pour une transaction acceptée : débite l'émetteur (valeur + effort), incrémente son nonce, puis retourne le nouvel état. **CreditReceiver** Crédite le récepteur dans `State`. S'il n'existe pas, il est ajouté avec balance: 0 et nonce: 0, puis la valeur de la transaction est ajoutée à son solde.

1.3 Choix de conception

Traitement récursif et utilisation d'accumulateurs pour contourner l'immutabilité d'Oz - on aurait pu aussi utiliser des cellules, mais on y a seulement pensé en rédigeant le rapport. Validation de chaque transaction avant son application. Utilisation d'un état temporaire par bloc (`CurrTmpState`) permettant de gérer les transactions bloc par bloc et de les appliquer une à une sans compromettre l'état global. Création d'une fonction auxiliaire `FinalizeCurrentBlock` afin de décomposer le travail, permettant à la fois de vérifier la validité du bloc et de préparer les accumulateurs pour le bloc suivant. Utilisation de la fonction `CreditReceiver` pour gérer le cas où le récepteur n'est pas encore présent dans l'état.

2 État final de la blockchain

2.0.1 État final des utilisateurs

Utilisateur	Balance	Nonce
11	97 415	2
14	70 294 080	8
15	94 650 918	6
32	5 141 804	7
37	1 239 534 151	0
49	463 119	1
60	36 160	3
61	221 043	4
66	480 590	2
73	58 497	4
86	918 173	2
87	210 757	1
100	579 432	3

2.0.2 Blockchain finale

BlocNum	Hash précédent	Hash	Transactions
0	0	256424	[22472, 38896, 81865, 61514, 51677]
1	256424	153662	[363117, 465569, 68551]
2	153662	146614	[751304, 50438, 62480, 128728]
3	146614	647331	[291119, 170096, 24257, ...]
4	647331	102336	[247790, 139871, ...]
5	102336	272434	[92750, ...]
6	272434	551027	[...]
7	551027	297173	[...]

2.0.3 Phrase secrète

“prof peter van royiarty is behind oz”

3 Usage de l’IA

Pour la rédaction du rapport, nous avons uniquement utilisé l’IA pour la correction orthographique et pour la formulation de certaines phrases. Mathéo a utilisé l’IA dans l’exécution de la blockchain pour s’informer sur les fonctions intégrées en Oz et pour le décodage du code à la fin, lors du make run, en cas de blocage important.

4 Extension 1: Denylist

4.0.1 Implémentation

Nous avons choisi, pour cette extension, de nous appuyer sur deux nouveaux records : BlockCounter, qui va compter le nombre de transactions de chaque utilisateur par bloc, et Denylist, qui va stocker les utilisateurs ayant envoyé trois transactions ou plus dans un même bloc.

Le fonctionnement est le suivant : BlockCounter est un record temporaire qui se réinitialise à chaque nouveau bloc. Pour chaque transaction (valide ou non) au sein d'un même bloc, on incrémente le nombre de transactions de l'utilisateur en question. Si ce nombre atteint 3, on l'ajoute à Denylist.

Avant toute validation et exécution d'une transaction, on vérifie si l'utilisateur est dans Denylist ; si c'est le cas, on ignore la transaction et elle n'est pas prise en compte.

4.0.2 État final des utilisateurs et blockchain

État final des utilisateurs

Utilisateur	Balance	Nonce
11	91551	0
14	70226791	8
15	94083260	5
32	6168760	2
37	1239534151	0
49	463119	1
60	36160	3
61	221043	4
66	457435	2
73	64726	2
86	839937	2
87	210757	1
100	288449	3

Blockchain finale avec Denylist

Bloc	Prev	Hash	Transactions
0	0	194910	[22472, 38896, 81865, 51677]
1	194910	92148	[363117, 465569, 68551]
2	92148	85100	[751304, 50438, 62480, 128728]
3	85100	294698	[170096, 24257, 15242]
4	294698	682363	[247790, 139871]
5	682363	852461	[92750, ...]
6	852461	106549	[...]
7	106549	735594	[...]

4.0.3 Phrase secrète

jcaiebat ovnqbooasc ikinfrifwvdbp

5 Extension 2: L'effort n'est pas gratuit

5.0.1 Implémentation

Pour cette extension, nous avons décidé d'utiliser les fonctions `ValidationTransactionWithState` et `ApplyTransaction`. Pour la première, le solde doit maintenant être supérieur à la valeur de la transaction ET au coût de l'effort. Pour la deuxième, on retire désormais la valeur de la transaction ET le coût de l'effort.

5.0.2 État final des utilisateurs (extension Effort)

Utilisateur	Balance	Nonce
11	97353	2
14	70293640	8
15	94650668	6
32	5141523	7
37	1239534151	0
49	463088	1
60	36035	3
61	220919	4
66	480496	2
73	58341	4
86	918127	2
87	210726	1
100	579275	3

5.0.3 Blockchain finale avec le coût de l'effort

Bloc	Prev	Hash	Transactions
0	0	256424	[22472, 38896, 81865, 61514, 51677]
1	256424	153662	[363117, 465569, 68551]
2	153662	146614	[751304, 50438, 62480, 128728]
3	146614	647331	[291119, 170096, 24257, ...]
4	647331	102336	[247790, 139871, ...]
5	102336	272434	[92750, ...]
6	272434	551027	[...]
7	551027	297173	[...]

5.0.4 Phrase secrète

prof peter van royiarty is behind oz